Does Deterministic Generate–Validate–Repair Reduce Unsafe LLM-Generated Network Changes? A Confirmatory Paired Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

***Abstract*—We preregistered and executed a sealed paired confirmatory study (n = 200 intent→configuration tasks) to test whether a deterministic generate–validate–repair (GVR) pipeline that consults a deterministic NetOps validator reduces validator-detected unsafe or semantically incorrect network changes relative to an LLM-only generation pipeline. The run used a single hosted model (gpt-5-mini) with adapter-enforced shared-initial generation semantics and a primary deterministic validator (deterministic_netops_validator_v5). Execution completed all planned pairs (completed_pairs = 200) consuming 200 model calls (one shared initial generation per task); no repair calls were issued because every initial candidate passed the validator. Paired contingency: n11 = 200, n10 = 0, n01 = 0, n00 = 0; estimate = 0.0; exact McNemar two-sided p = 1.0; 95% bootstrap percentile CI = [0.0, 0.0] (10,000 resamples, seed = 1729). Because the baseline validator-detected unsafe incidence was 0% on this task bank, the preregistered >=50% relative-reduction hypothesis could not be meaningfully evaluated. We report the sealed design, execution accounting (start: 2026-08-30T11:50:34.530059Z; end: 2026-08-30T12:12:07.541650Z), the Mininet 10% holdout (20 tasks) that produced zero validator–emulator disagreements, and an artifact-grounded discussion of operational implications and threats to validity (preregistration id: cnsms2026-gvr-effectiveness-02).**

I. Introduction

Automating intent→configuration translation with generative language models can increase NetOps productivity but risks producing incorrect or unsafe changes that cause outages or policy violations. Prior work therefore proposes grounding, deterministic validation, and repair loops (generate–validate–repair, GVR) to detect and correct unsafe candidates before application. The operational benefit of automated repair is conditional on three factors: (i) baseline prevalence of validator-detectable failures from the chosen generator/prompting policy; (ii) validator coverage and fidelity; and (iii) the available repair budget and repair-prompt effectiveness. We preregistered a narrowly scoped confirmatory question: Does an automated, deterministic GVR pipeline that consults a deterministic NetOps validator materially reduce validator-detected unsafe or semantically incorrect network changes relative to an LLM-only generation pipeline on a curated 200-task intent→configuration bank? The study was designed as a paired experiment that fixes the generator output per task and isolates the marginal effect of invoking a validator+repair on the same candidate.

II. Related Work

Recent literature documents three relevant threads for LLM-driven NetOps: (1) empirical characterizations of LLM factuality failures and mitigation patterns, (2) NetOps-specific prototypes that translate intent into configurations, and (3) system and threat analyses that focus on verification, guarded architectures, and attack surfaces in agentic tool use. First, cross-domain studies and surveys show that large language models produce factual errors, omissions, and hallucinations in operational tasks and that grounding, verifier-assisted repair, and uncertainty estimators can reduce but not eliminate such errors (e.g., Farquhar et al.; broad LLM surveys) [https://openalex.org/W4399803256, 10.1007/s11704-026-60308-3]. These works establish that mitigation effectiveness depends strongly on prompt design, sampling policy, retrieval/grounding fidelity, and task framing. Second, a growing set of NetOps prototypes demonstrates the feasibility of intent→configuration generation but also highlights evaluation gaps: NetConfEval and several intent-driven configuration works show that LLMs can synthesize network commands at prototype scale, yet reported evaluations often lack production-like instrumentation, end-to-end emulation, or preregistered inference procedures for failure-rate estimates [https://openalex.org/W4399629579, https://openalex.org/W4403635865, https://openalex.org/W4400072049]. Third, architectural proposals and empirical audits advocate combining generation with deterministic validators or simulators (generate–validate–repair, Batfish-style checks, and Mininet emulation) and propose bounded-autonomy guardrails to constrain unsafe agentic actions; at the same time, red-team and configuration-brittleness studies document that tool descriptions, chaining ambiguity, and persona/history can materially change generated actions and enable exploits if guardrails are incomplete [https://openalex.org/W4410125989, 10.36227/techrxiv.177004277.71795055/v1, 10.2139/ssrn.7203631, 10.21203/rs.3.rs-10646209/v1]. Complementary work on prompt sensitivity and LLM non-determinism shows that evaluation outcomes can shift with temperature, sampling, system prompts, or chain-of-thought strategies, which complicates comparisons across pipelines and motivates paired or shared-initial designs for controlled measurement

[https://openalex.org/W4402860127]. Finally, proposals for secure-by-default agent patterns and bounded-autonomy frameworks emphasize runtime veto layers, explicit tool registries, and staged validation to limit unsafe changes, but these remain largely proof-of-concepts whose real-world efficacy depends on validator coverage and emulation fidelity [10.36227/techrxiv.177006109.97957916/v1, 10.36227/techrxiv.177004277.71795055/v1]. Synthesizing these threads reveals a persistent evaluation gap: few studies provide preregistered, externally auditable measurements that isolate the marginal effect of deterministic repair on a fixed generator output under controlled sampling semantics. Our preregistered paired design addresses precisely this gap by enforcing shared-initial generation and deterministic validator checks, thereby separating generator correctness under a fixed prompt policy from any additional gains attributable to automated repair.

III. Methodology

Preregistration and inferential plan. We froze the experimental plan under preregistration id cnsms2026-gvr-effectiveness-02 before any model calls. The confirmatory design was paired and deterministic: n = 200 tasks (four topology clusters $\times$ 50 tasks), one shared deterministic initial LLM generation per task, and a repair budget of at most one automated repair call per task. The primary estimand was the paired_success_rate_difference_guarded_minus_baseline and the prespecified test was an exact two-sided McNemar test (α = 0.05). Confidence intervals for the paired difference were computed by the bootstrap percentile method with 10,000 resamples and bootstrap_seed = 1729. The analysis executor was paired_binary_analysis_v1 as recorded in the analysis artifacts.

Adapter, model, and sampling semantics. The run used a single hosted model declared as gpt-5-mini (provider label: openai_responses). Model configuration metadata (execution/model_configuration.json) records requested_model = "gpt-5-mini", max_output_tokens = 2000, maximum_attempts_per_call = 1, and temperature = null (deterministic sampling enforced by the adapter). Crucially, the hosted_netops_gvr_v1 adapter enforced shared_initial generation semantics: for each task the system generated exactly one initial candidate which was deterministically reused for both the baseline (LLM-only) and guarded (GVR) conditions. This shared-initial design eliminates per-condition sampling variance and isolates the incremental effect of invoking validation and an allowed repair step on the identical generator output. Adapter accounting in deterministic_reconciliation.json documents hosted_model_call_event_count = 200, stage_counts = {shared_initial_generation: 200}, and model_calls_used = 200, confirming the enforced single shared invocation per task and that no separate condition-specific sampling occurred.

Task bank, validators, and holdout emulator. The task bank is a curated, machine-checkable intent→configuration set (200 tasks) in which each task encodes initial and expected final states plus workflow policies (e.g., make_before_break_uplink_switchover, must_be_down_for_fields). The primary deterministic checker (deterministic_netops_validator_v5, validator_version = "5.0") runs syntactic parsing, intent/constraint enforcement, and structured diagnostics (fields available in validator traces include valid flag, violation_count, violation_codes, parsed_command_count, required_command_count, and difficulty annotations). A Mininet-based data-plane emulator was preregistered as a secondary disagreement detector applied to a randomized 10% holdout (20 tasks) to detect validator–emulator mismatches; the executed contamination_summary artifact reports zero disagreement flags for that holdout. The preregistration specified deterministic treatment rules: pairs flagged for validator–emulator disagreement would be excluded from the primary analysis per the contamination_plan; sensitivity analyses for alternative treatments were preregistered but not invoked because no flags occurred.

Repair policy, budgeting, and missingness handling. Per the preregistration, when the primary validator reported violations the pipeline was permitted to issue up to one automated repair prompt to the same LLM for that task (maximum repair calls across the sample = 200). Repair prompts, if used, would be captured in repair_provider_trace fields in the episode artifacts; none appear in the archived episodes because no repairs were applied. Missingness and exclusion rules were deterministic: pairs where neither condition completed because of infrastructure failures would be excluded and logged; pairs with contamination flags (validator–emulator disagreement) would be excluded from the primary confirmatory test. The execution manifest and missingness_summary show complete_pair_count = 200 and zero failed episodes, so the planned missingness handling was not invoked in practice.

Execution accounting and artifact capture. The autonomous run executed between 2026-08-30T11:50:34.530059Z and 2026-08-30T12:12:07.541650Z. The adapter enforced the preregistered maximum model calls (maximum_model_calls = 400) and per-task caps (initial generation = 1 call; repair calls <= 1). The run consumed 200 model calls (one shared initial generation per task), with cache_miss_count = 200 and cache_hit_count = 0 as recorded in the deterministic reconciliation artifacts. All model calls, prompts, raw responses, validator traces, provider traces, and analysis artifacts (execution/raw_results.jsonl, analysis/paired_contingency_table.csv, analysis/condition_summary.csv, analysis/contamination_summary.csv, analysis/analysis_log.jsonl, execution/model_configuration.json) were archived; representative per-task artifacts (e.g., execution/scoring/task-000001-baseline.json) show validator_trace fields and scoring outcomes. The analysis pipeline executed the exact McNemar test and bootstrap CI generation exactly as preregistered; analysis/analysis_log.jsonl records bootstrap_resamples = 10000 and bootstrap_seed = 1729.

Design rationale and implications for internal validity. The

shared_initial paired design was chosen to isolate the pure marginal effect of validation+repair on a fixed generator output; this reduces variance attributable to stochastic sampling and makes the confirmatory test focused on whether repair can convert a validator-failing candidate into a validator-passing one for the same initial output. The trade-off is explicit: the design sacrifices generality across multiple independent generator draws for stronger internal control. Preregistration, deterministic adapter semantics, and frozen stopping rules were used to avoid post-hoc analytic choices. Deterministic reconciliation artifacts confirm pair_accounting_consistent = true and all_deterministic_consistency_checks_passed = true.

Limitations baked into the methodology. The methodology intentionally restricts scope: a single model, a deterministic prompt/sampling policy, a one-repair budget, and a curated task bank. These constraints reflect an explicit confirmatory question about marginal GVR effect under a tightly controlled generator policy rather than an evaluation of alternative sampling, multi-step repair strategies, or cross-model generality. When interpreting outcomes, readers should account for these preregistered methodological choices documented in the archived plan.

IV. Results

Execution completeness and basic counts. The experiment executed to completion under the preregistered stopping rule. Accounting artifacts report completed_episode_count = 400 (shared initial generation reused across conditions), complete_pair_count = 200, failed_episode_count = 0, and model_calls_used = 200 (one shared initial generation per task). The adapter/provider audit recorded hosted_model_call_event_count = 200, cache_miss_count = 200, stage_counts = {shared_initial_generation: 200}, and condition_counts = {shared: 200}, confirming that no separate per-condition re-sampling occurred.

Validator outcomes and paired contingency. The primary deterministic validator (deterministic_netops_validator_v5) marked every shared initial candidate as valid. The condition summary artifact (analysis/condition_summary.csv) contains: "baseline,200,0,200,1.0" and "guarded,200,0,200,1.0" (success_rate = 1.0 for both conditions). The paired contingency artifact (analysis/paired_contingency_table.csv) records the contingency table used in the exact McNemar test as: - n11 (pass/pass) = 200 - n10 (guarded pass, baseline fail) = 0 - n01 (guarded fail, baseline pass) = 0 - n00 (fail/fail) = 0

Inferential result and bootstrap CI. With zero discordant pairs the observed paired difference estimate = 0.0. The pre-registered exact McNemar two-sided test returns p = 1.0. The bootstrap percentile 95% confidence interval computed with 10,000 resamples and bootstrap_seed = 1729 is [0.0, 0.0]. These numerical outputs are recorded in analysis/results.json and the analysis/analysis_log.jsonl artifact (bootstrap_resamples = 10000, bootstrap_seed = 1729).

Repair activity and secondary outcomes. The preregistered repair budget allowed up to 200 repair calls (one per task) but no repair calls were issued because no initial candidate failed the validator. The secondary_results array is therefore empty and the intended secondary estimands (average repair iterations, GVR-introduced failure rate, model-call cost per successful repair) are unpopulated in the analysis bundle. The analysis logs explicitly note repair_calls_used = 0.

Representative per-task diagnostics. Representative scoring artifacts for sample pairs illustrate validator traces and complexity indicators while still producing validator-passing configurations. Examples taken from archived representative tasks: - pair-000001: validator_trace.validation_after.valid = true; violation_count = 0; parsed_command_count = 4; required_command_count = 4; difficulty.level = "medium"; pattern = "shutdown_configure_restore"; required_sequence length = 4. - pair-000002: validator_trace.validation_after.valid = true; violation_count = 0; parsed_command_count = 2; required_command_count = 2; difficulty.level = "hard"; pattern = "make_before_break_uplink_switchover". - pair-000003: validator_trace.validation_after.valid = true; violation_count = 0; parsed_command_count = 6; required_command_count = 6; difficulty.level = "hard"; pattern = "paired_link_atomic_mtu_change". These artifacts demonstrate that tasks exercised medium-to-very_hard workflow constraints (examples in the archive include labels "medium", "hard", and "very_hard") and that the model outputs conformed to the task-encoded required_sequences and validator checks despite nontrivial required_command_counts.

Emulator (contamination) check. Per the preregistered contamination_plan, a Mininet-based data-plane emulator was run on a randomized 10% holdout (20 tasks). The analysis/contamination_summary.csv artifact records zero disagreement flags between deterministic_netops_validator_v5 and the Mininet emulator on that holdout (flag counts empty). Deterministic reconciliation artifacts further report pair_accounting_consistent = true and all deterministic consistency checks passed. We note, however, that the explicit labeled seed for the randomized holdout selection is not present among the labeled artifacts; the contamination_summary artifact nevertheless documents that the secondary emulator check executed and found no disagreements for the recorded holdout.

Unexecuted sensitivity branches and their artifact evidence. Several preregistered sensitivity analyses and contingency branches (e.g., treating flagged tasks as failures or successes, alternative missingness treatments, and inclusion of flagged tasks in the primary analysis) were not invoked because no tasks were flagged and no episodes failed. The execution manifest and analysis artifacts explicitly record these branches as unused; the analysis/contamination_summary.csv and analysis/missingness_summary.csv reflect zero flags and zero missing pairs.

Operationally relevant diagnostics. The run produces three operationally meaningful, artifact-grounded diagnostics: (1) baseline validator failure prevalence in this curated workload was 0% (200/200 passes) under the chosen generator/prompting policy and sampling semantics; (2) the adapter enforced shared_initial semantics eliminated

per-condition sampling variance, so the absence of discordant pairs is not attributable to sampling differences across conditions but to the generator outputs themselves; and (3) the Mininet holdout found no validator–emulator mismatches on the 20-task sample, providing limited end-to-end corroboration of validator pass verdicts for that holdout. All of these points are directly supported by the archived artifacts referenced above (execution/model_configuration.json, deterministic_reconciliation.json, analysis/condition_summary.csv, analysis/paired_contingency_table.csv, analysis/contamination_summary.csv, and representative task scoring artifacts).

V. Discussion

Conditional interpretation and statistical mechanics. The confirmatory null (zero baseline validator failures) is an artifact-grounded outcome: baseline_success_rate = 1.0 (200/200) and guarded_success_rate = 1.0 (200/200), producing a paired contingency with n11 = 200 and no discordant cells. In paired binary inference this configuration is degenerate for McNemar's test—the exact two-sided p = 1.0 and the bootstrap percentile 95% CI = [0.0, 0.0] (bootstrap parameters: resamples = 10,000, seed = 1729). Those numbers are not evidence that repair is useless generally; they are a precise statement that, on this curated workload and under the frozen shared-initial generation policy, there were no validator-detectable failures for the GVR pipeline to reduce. The key interpretive point is therefore conditional: the marginal utility of automated repair is a function of baseline validator failure prevalence and validator coverage for the targeted workload.

Artifact-grounded diagnostics explaining the null. Several archived artifacts document why initial candidates passed at scale. First, each task encodes explicit required_sequence and workflow constraints; the primary validator checks those constraints and reports parsed_command_count and required_command_count for each task (representative traces show required_command_count values matching parsed_command_count). Second, difficulty annotations in validator traces (labels such as "make_before_break_uplink_switchover", "paired_link_atomic_mtu_change") show that the task bank exercised nontrivial dependency patterns even when outputs were correct; correctness under these labels indicates the generator/prompt pair produced sequences that satisfied the machine-checkable constraints. Third, adapter accounting (deterministic_reconciliation) shows that hosted_model_call_event_count = 200 and stage_counts = {shared_initial_generation: 200}, confirming a single deterministic generation per task was shared across conditions—this design choice removed sampling variability as a pathway to observe alternative (potentially failing) candidates. Fourth, the preregistered Mininet secondary check ran on a randomized 10% holdout (20 tasks) and the contamination summary (analysis/contamination_summary.csv) records zero disagreements between the primary validator and emulator; this artifact verifies that, in the holdout sample, validator verdicts matched the emulator's checks as executed in this run. We also disclose a reproducibility gap: the labeled artifact set does not include an explicit selection seed for the Mininet holdout, which prevents bit-for-bit reconstruction of the exact holdout selection even though the contamination summary reports zero disagreements.

Methodological tradeoffs: shared-initial pairing versus unconditional failure measurement. The shared_initial paired design was chosen to isolate the incremental effect of invoking validator+repair on the same generator output. That isolation gains internal control at the cost of observing only the conditional effect on a fixed sample: it cannot capture gains obtainable by re-sampling the LLM (e.g., producing alternative candidates that a repair could act upon) or by employing nonzero temperature sampling to elicit diverse failure modes. For practitioners or evaluators whose goal is to estimate unconditional operational failure rates (the risk of an LLM pipeline producing an unsafe candidate on any application), a design with independent per-condition sampling and multiple draws per task will produce better estimates of unconditional baseline failure prevalence and of the repair mechanism's ability to convert some failing draws into passes. Our artifact set supports both measurements in principle, but this confirmatory run intentionally prioritized a paired, shared-initial comparison per the preregistered estimand.

Validator coverage and emulator corroboration. Deterministic validators are necessary but not sufficient oracles for operational safety: they codify a set of syntactic and policy checks (here, deterministic_netops_validator_v5) that map to task-bank constraints. The Mininet emulator was included as a pragmatic secondary detector to reveal validator blind spots that manifest at the data-plane level; the zero disagreements observed in the 20-task holdout demonstrate no such blind spots were detected in that sample for this run, but holdout scale and the missing labeled selection seed limit how strongly that result can be generalized. A robust operational assurance program should therefore combine diverse validators (vendor-specific semantics, richer device models), larger emulation or testbed sampling, and adversarial/perturbation tests to probe validator blind spots; the artifacts here illustrate where such secondary checks succeeded (no disagreements) and where exact reproducibility of the holdout draw is currently unverifiable (missing seed).

Design implications for guardrail engineering and operational deployment. The neutral outcome suggests practitioners evaluate two complementary axes before investing heavily in automated repair: (1) measure baseline validator failure prevalence on representative workloads under the intended generation policy; (2) assess validator completeness via emulation or selective real-device checks. If baseline failures are already rare under the production generator/prompt policy, then (a) a single-pass repair budget delivers limited marginal safety returns for validator-detected failures, and (b) resources may be better allocated to expanding validator fidelity, adversarial

testing, or introducing multi-sample generation and model ensembles to surface latent failure modes. Conversely, when baseline failures are nontrivial, GVR with an appropriately sized repair budget and well-designed repair prompts can reduce validator-detected errors; the archived preregistration captures how repair budgeting and maximum repair calls were constrained here (up to one repair call per task), and the absence of repair events means the exploratory statistics for repair iterations and model-call costs remained empty in the analysis artifacts.

Threats to validity and concrete artifact links. Internal validity is strengthened by preregistration, deterministic adapter enforcement, and deterministic reconciliation artifacts that confirm episode accounting (complete_pair_count = 200, model_calls_used = 200). Construct validity remains conditional on validator expressiveness: representative validator_trace fields (validator_id, validator_version, violation_count, parsed_command_count) show what the validator measured, but validators do not represent all device idiosyncrasies. External validity is limited by the single hosted model (gpt-5-mini), the shared_initial deterministic sampling policy, and the curated task bank; these constraints are recorded in execution/model_configuration.json and the preregistration artifact. Conclusion validity: with zero discordant observations the primary confirmatory test is uninformative about the preregistered 50% relative reduction; the observed numbers are nonetheless exact and reproducible within the archived bundle (aside from the Mininet-holdout seed gap noted above).

Practical recommendations for future confirmatory studies. To measure GVR effectiveness where baseline failure prevalence is low, future preregistered runs should (i) adopt a mixed design that includes independent per-condition multi-sample draws to estimate unconditional failure rates; (ii) increase the repair budget and allow multi-step repairs or human-in-the-loop escalation to exercise more repair pathways; (iii) scale secondary emulation checks and record selection seeds for holdouts to enable exact reproducibility; and (iv) evaluate multiple models or ensembles to probe model-dependent brittleness. The archived artifacts and the deterministic reconciliation logs provide a template for implementing these changes while preserving auditability.

VI. Limitations

- Task-bank scope: the confirmatory sample used a curated synthetic/public intent→configuration bank (n = 200); results therefore reflect this bank and may not generalize to broader production workloads or adversarial distributions.
- Single-model and shared-initial setting: only gpt-5-mini (hosted) with adapter-enforced shared-initial generation was tested (execution artifacts record hosted_model_call_event_count = 200 and stage_counts shared_initial_generation = 200). Outcomes may differ across model families, independent per-condition sampling, nonzero temperature sampling, or larger repair budgets.
- Validator coverage and oracle limitations: the primary deterministic validator enforces a defined set of syntax/intent constraints; validators can fail to capture real device idiosyncrasies, vendor-specific behaviors, or emergent failure modes detectable only by end-to-end deployment.
- Adversarial robustness untested: this confirmatory run did not include active adversarial injection or red-team tool-description attacks; prior audits indicate such vectors can bypass naive defenses and remain a separate concern.
- Reproducibility gap for contamination check: the randomized holdout selection seed for the Mininet contamination check is absent from the labeled artifacts, preventing exact reconstruction of the holdout selection.
- Single repair budget: the GVR pipeline allowed at most one automated repair prompt per task; multi-step repair strategies, different repair budgets, or human-in-the-loop approvals were not exercised here.

VII. Conclusion

We preregistered and executed a sealed paired study comparing LLM-only generation to a deterministic GVR pipeline on 200 NetOps intent→configuration tasks using gpt-5-mini and deterministic_netops_validator_v5. Both pipelines produced validator-passing configurations for every task (n11 = 200; no discordant pairs), resulting in an observed paired difference of 0.0, exact McNemar p = 1.0, and bootstrap 95% CI = [0.0, 0.0]. The preregistered >=50% relative-reduction hypothesis could not be evaluated because baseline validator failures were absent. The artifact-grounded outcome clarifies that GVR's marginal value is workload dependent: when baseline validator-detectable failures are rare, automated repair yields no measured benefit for those validator-detected errors. Future studies should target workloads with nontrivial baseline failure rates, expand validator fidelity with richer emulation or real-device checks, evaluate multi-sample and multi-model policies, and explore multi-step or human-in-the-loop repair budgets to characterize practical operational gains. All execution artifacts and analysis outputs are archived for independent inspection under the preregistration id cited above.

Disclosure Statement

Disclosure (mandatory). This study executed under the autonomous post-lock preregistration cnsms2026-gvr-effectiveness-02. Declared model: gpt-5-mini (provider label: openai_responses) invoked via the hosted_netops_gvr_v1 adapter. The exact initial master prompt is archived at provenance/master_prompt.txt (SHA-256: 1872df1e1805d2d96940456ca016bd665d1d5196 add77f5acdf1582bb39b15ba). Topic constraints: the human Research Director limited the study to Generative AI and LLMs for NetOps focusing on safety/reliability of generated network changes. Frameworks and tools: orchestration used hosted_netops_gvr_v1 and the deterministic task generator deterministic_netops_task_generator_v5. Primary deterministic validator: deterministic_netops_validator_v5

(intent/constraint checks). A Mininet-based emulator was preregistered and executed as a secondary disagreement detector on a randomized 10% holdout (20 tasks); the artifact analysis/contamination_summary.csv shows zero disagreements for that holdout. Preregistration and freeze: the experimental plan (estimand, sample size n = 200, paired design, stopping rules, max model calls = 400, repair budget = 1 per task, shared-initial generation semantics) was frozen before any model calls. Autonomous execution and artifacts: the run executed autonomously under the frozen plan (start: 2026-08-30T11:50:34.530059Z; end: 2026-08-30T12:12:07.541650Z); model calls, prompts, seeds, validator traces, emulator traces, logs, and the artifact manifest are archived in the execution bundle. Model-call budget and usage: 200 model calls were used (one shared initial generation per task); up to 200 repair calls were allowed but none were applied because initial candidates passed the validator. Human involvement: the human Research Director selected the topic family and pre-lock constraints; no human manual editing or selective rewriting of technical manuscript content occurred after the autonomy lock. Deviations and restarts: no post-preregistration design changes or technical restarts occurred. Known reproducibility gap: the explicit randomized selection seed used to draw the Mininet 10% holdout is not present as a labeled artifact in the archive; this absence prevents exact deterministic reconstruction of the drawn holdout despite the contamination summary reporting no disagreements. The master prompt archive reference above is the immutable prompt required by the track.

REFERENCES

[1] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Zican Dong, Yupeng Hou, Beichen Zhang, Yingqian Min, Junjie Zhang, Peiyu Liu, Xiaolei Wang, Yifan Du, Yushuo Chen, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Peiyu Liu, Yiwen Hu, Jian-Yun Nie, Ji-Rong Wen, "A Survey of Large Language Models", 2026, 10.1007/s11704-026-60308-3

[2] Changjie Wang, Mariano Scazzariello, Alireza Farshin, Simone Ferlin, Dejan Kostić, Marco Chiesa, "NetConfEval: Can LLMs Facilitate Network Configuration?", 2024, 10.1145/3656296

[3] Oscar G. Lira, Oscar Maurício Caicedo Rendón, Nelson L. S. da Fonseca, "Large Language Models for Zero Touch Network Configuration Management", 2024, 10.1109/mcom.001.2400368

[4] Chirag Shah, Ryen W. White, Reid Andersen, Georg Buscher, Scott Counts, Sarkar Snigdha Sarathi Das, Ali Montazeralghaem, Sathish Manivannan, J. Neville, Nagu Rangan, Tara Safavi, Siddharth Suri, Mengting Wan, Leijie Wang, Longqi Yang, "Using Large Language Models to Generate, Validate, and Apply User Intent Taxonomies", 2025, 10.1145/3732294

[5] omar altawil, Dr. Mustafa Al-Fayoumi, "Configuration-Level Semantic Brittleness in Tool-Using LLM Agents: A Factor-Ranking Study of Persona, History, and Tool Definition", 2026, 10.2139/ssrn.7203631

[6] Mohammadreza Rashidi, "Measuring How LLM Tool Descriptions, Cross-Tool Ambiguity, and Action Chains Compose into Excessive Agency Exploits", 2026, 10.21203/rs.3.rs-10646209/v1

[7] Yoshihiro Ando, "Secure-by-Default Guardrails for MCP-Based Tool Use in Multi-Modal LLM Agents", 2026, 10.36227/techrxiv.177006109.97957916/v1

[8] Sebastian Farquhar, Jannik Kossen, Lorenz Kuhn, Yarin Gal, "Detecting hallucinations in large language models using semantic entropy", 2024, 10.1038/s41586-024-07421-0

[9] Shuyin Ouyang, Jie M. Zhang, Mark Harman, Meng Wang, "An Empirical Study of the Non-Determinism of ChatGPT in Code Generation", 2024, 10.1145/3697010

[10] Oghenekeno Hilkiah Okula, ""The Era of AI Agents for Network Engineering": Intent-Aware Auto Remediation for Network Disruption or Failures Using AI Agents, Large Language Models (LLM) and Model Context Protocol (MCP) Mechanisms", 2026, 10.36227/techrxiv.177004277.71795055/v1